

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 2

Duration : 1 Hour

Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I M Mohan siva reddy/192573011(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

msvvreddy

Annexure A

Constraint-Based Problem Solving

Annexure A

Student 23: M Mohana Siva Vikasvanth Reddy | Register Number: 192573011

1.Scenario

A firefighting robot is deployed inside a burning multi-storey building with an unknown internal layout. It must: navigate safely to locate trapped occupants, avoid hazards (collapsed structures, fire zones), operate with limited visibility due to smoke, and make decisions without a complete floor plan. The robot updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search.
- ii. Analyze the challenges of partial observability and dynamic environments.
- iii. Describe how the robot builds and updates its internal model.
- iv. Compare online search with uninformed and informed search strategies.
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety.

2. Scenario

An airport is designing an AI system to automatically assign arriving flights to available gates. The system must satisfy multiple constraints: no two flights can occupy the same gate at overlapping times, a limited number of gates support wide-body aircraft, certain airlines have contractual gate preferences, ground crew availability constraints, and special accommodations for delayed connecting flights. The complexity increases as the number of flights and gates grows.

Tasks: Formulate the problem as a Constraint Satisfaction Problem (CSP).

- i. Clearly define variables, domains, and constraints with explanation.
- ii. Explain how backtracking search works in this scenario.
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency.
- iv. Analyze real-world challenges and justify the best strategy for scalability.

3.Scenario

A board-game company is developing an AI opponent for a turn-based strategy game similar to checkers, to be deployed on low-power tablets. The AI must compete against human players, make decisions within strict time limits, evaluate complex board states with multiple possible moves, and the decision tree is extremely large, making computation expensive on limited hardware.

Tasks:

- i. Explain how the Minimax algorithm models this problem.
- ii. Analyze the computational challenges of large game trees.
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.
- iv. Design an evaluation function and justify the factors considered.
- v. Recommend a strategy for balancing optimality and real-time performance.

1. Firefighting Robot — Online Search Agent

i. Why Online Search Instead of Offline Search

Offline search (e.g., plain A*, BFS/DFS) requires a **complete, known state space** before computing a solution — the whole map must be available upfront, and the environment must stay static while the plan executes. Here neither holds:

- The building layout is **unknown in advance** — nothing exists to search over until the robot explores.
- Fire, smoke, and structural collapse **change the environment during execution**, so a plan computed once can become invalid before it finishes.
- The robot must **sense-act-replan** continuously rather than "think first, act later."

An **online search agent** interleaves acting and planning: take an action, observe the outcome, update the internal map, then decide the next action. This is the only viable model when the state space is discovered incrementally and conditions shift in real time.

ii. Challenges of Partial Observability and Dynamic Environments

Partial observability:

- Sensors (thermal, LIDAR, gas) only cover a **local range**, further degraded by smoke.
- The robot cannot always distinguish between states that look sensorially similar (perceptual aliasing).
- It must maintain a **belief state** (probability distribution over possible situations) rather than assume a single known state.

Dynamic environment:

- Fire zones expand and structural integrity degrades **over time**, so "safe" areas can become hazardous mid-mission.
- Previously gathered knowledge has a **decay window** — old data cannot be trusted indefinitely.
- The agent must continually re-verify assumptions rather than treat its map as fixed truth.

Together, these push the problem toward a **POMDP-like** (partially observable, non-deterministic, dynamic) formulation rather than a simple deterministic graph search.

iii. How the Robot Builds and Updates Its Internal Model

1. **Sensing:** LIDAR/sonar/thermal/gas sensors gather local readings each step (compensating for poor visibility).
2. **Incremental mapping (SLAM-style):** New readings are fused with the existing map to build an **occupancy grid** — free, occupied, hazardous, or unknown cells.

3. **Hazard tagging with decay:** Detected fire/collapse zones get a confidence score that **decreases over time**, forcing re-verification.
4. **Belief update:** A Bayesian filter (occupancy-grid update or particle filter) revises hazard/occupant probabilities as evidence accumulates.
5. **Re-planning trigger:** Significant map changes (new hazard, blocked path) trigger a fresh local search over the currently known region.

iv. Online Search vs Uninformed vs Informed Search

Aspect	Uninformed (BFS/DFS)	Informed (A*/Greedy)	Online Search
Requires known map	Yes	Yes	No — built incrementally
Planning timing	Fully before execution	Fully before execution	Interleaved with execution
Adapts to change	No	No	Yes
Uses heuristics	No	Yes	Optional (e.g., LRTA*)
Handles partial observability	No	No	Yes, by design
Suitability here	Poor	Poor	Necessary

Both classical strategies assume the search space is inspectable before commitment — a luxury unavailable in an unmapped, burning building. **LRTA* (Learning Real-Time A*)** blends a heuristic-guided informed approach with incremental, online discovery.

v. Justified Recommendation

An **online, informed, real-time search approach** (LRTA*-style) layered with **belief-state tracking/SLAM** and **safety-constrained decision rules** is most suitable:

- **Uncertainty:** Bayesian belief updates handle noisy, partial sensing.
- **Adaptability:** Continuous re-planning responds instantly to newly discovered hazards.
- **Safety:** A heavily hazard-weighted cost function plus a "never move without a known safe retreat" rule prevents irreversible unsafe actions.

This combination balances goal-directed efficiency (finding occupants quickly) with survivability in an unmapped, worsening environment.

2. Airport Gate Assignment — CSP Formulation

i. Variables, Domains, and Constraints

Variables:

Let each arriving flight be a variable: $F_1, F_2, \dots, F_n$ (one variable per flight requiring a gate).

Domains:

For each flight F_i , the domain is the set of **compatible available gates**:

$\text{Domain}(F_i) = \{G_1, G_2, \dots, G_m\}$ restricted to gates that:

- Physically fit the aircraft type (wide-body vs narrow-body),
- Are operationally available during the flight's arrival/departure window.

Constraints:

Constraint	Type	Description
No time-overlap on same gate	Hard, binary	If F_i and F_j are assigned the same gate, their occupied time windows must not overlap
Wide-body compatibility	Hard, unary	Wide-body flights can only be assigned gates flagged wide-body-capable
Airline contractual preference	Soft/hard (configurable)	Certain airlines must (or should) use specific gates/terminals
Ground crew availability	Hard, binary/global	A gate assignment is invalid if no crew is available for that time slot
Connecting-flight accommodation	Soft, priority-based	Delayed connecting flights get preferential gate proximity/availability
Buffer time between uses	Hard, binary	Minimum turnaround time required between two flights on the same gate

Hard constraints define **feasibility** (must never be violated); soft constraints are optimized via a **weighted objective function** layered on top of the CSP (making this a **Constraint Optimization Problem**, technically, since some constraints are preferences rather than absolutes).

ii. How Backtracking Search Works Here

1. **Variable selection:** Pick an unassigned flight (commonly via **Minimum Remaining Values (MRV)** — the flight with the fewest compatible gates left, since it's most likely to fail and should be resolved early).
2. **Value assignment:** Try assigning a gate from that flight's domain (ordered by **Least Constraining Value** — the gate that rules out the fewest options for other flights).
3. **Constraint check:** Verify no violation — no time overlap, aircraft-type compatibility, crew availability.
4. **Recurse:** If consistent, move to the next unassigned flight.

5. **Backtrack:** If no gate in the domain satisfies constraints, undo the previous assignment and try the next alternative value for the prior flight.
6. **Terminate:** Success when all flights are assigned consistently; failure (no full assignment) triggers relaxation of soft constraints or escalation (e.g., manual override, remote parking stand).

iii. Constraint Propagation (Forward Checking) for Efficiency

Plain backtracking discovers conflicts **late** (only when reaching the conflicting variable), wasting search effort. **Forward checking** improves this:

- After assigning a gate to flight F_i , immediately **remove now-invalid gate options** from the domains of all *unassigned* flights that would conflict in time or compatibility.
- If any unassigned flight's domain becomes **empty**, backtrack immediately — no need to explore further down that branch.
- Stronger propagation (e.g., **Arc Consistency, AC-3**) can prune even more aggressively by ensuring every remaining domain value for one variable has a compatible value in every constrained neighbor, catching conflicts even before an assignment is attempted.

This drastically **shrinks the effective search tree**, especially important as flight/gate counts scale up.

iv. Real-World Challenges and Scalability Strategy

Challenges:

- **Scale:** Hundreds of daily flights $\times$ dozens of gates creates a large combinatorial space.
- **Dynamic updates:** Delays, cancellations, and new arrivals require **continuous re-solving**, not one-time solving.
- **Conflicting soft constraints:** Airline preferences vs crew availability vs connection accommodations may not all be satisfiable — needs a prioritized trade-off, not a strict CSP answer.
- **Real-time response requirement:** Full backtracking search from scratch on every disruption is too slow operationally.

Recommended strategy:

- Use **forward checking + MRV/LCV heuristics** to keep backtracking efficient at scale.
- Apply **local repair / incremental re-solving**: when a disruption occurs, only re-solve the affected subset of flights/gates rather than the entire schedule.
- Treat soft constraints via a **weighted penalty/objective function** layered on the CSP solver (turning it into a constrained optimization problem), so the system finds the best feasible compromise rather than failing outright when preferences conflict.
- Precompute a **robust baseline schedule offline** (with buffer margins) and use fast online constraint propagation for real-time adjustments — balancing solution quality against the need for near-instant responses.

3. Board-Game AI (Checkers-like) — Minimax on Low-Power Hardware

i. How Minimax Models This Problem

- **MAX (the AI):** chooses moves to maximize its evaluated advantage.
- **MIN (the human opponent):** assumed to play optimally against the AI, choosing moves to minimize the AI's advantage.
- **Game tree:** each node is a board state; edges are legal moves; each ply alternates between MAX and MIN.
- **Terminal nodes:** win/loss/draw states, or a depth cutoff evaluated heuristically.
- Minimax recursively backs up values from terminal/cutoff nodes: MIN nodes take the minimum of children's values, MAX nodes take the maximum — the AI selects the move leading to the best guaranteed outcome assuming a perfectly rational opponent.

ii. Computational Challenges of Large Game Trees

- **Exponential growth:** Tree size $\approx b^d$ (b = branching factor/legal moves, d = depth/pplies) — even checkers-like games have branching factors around 7–10, making full-depth search infeasible.
- **Memory constraints:** Low-power tablets have limited RAM/CPU, ruling out storing or fully exploring deep trees.
- **Time constraints:** Turn-based play still needs a response within a bounded time (seconds), forcing depth limits.
- **Energy/battery limits:** Exhaustive search drains battery quickly on mobile hardware, an added constraint beyond pure compute time.

iii. How Alpha-Beta Pruning Improves Efficiency (Detailed Reasoning)

- α = best value MAX can guarantee so far; β = best value MIN can guarantee so far.
- At a **MIN node**, if a child's evaluated value $\leq \alpha$, MAX already has a better alternative elsewhere in the tree, so the remaining children of this MIN node **cannot** change the outcome — prune them.
- Symmetrically, at a **MAX node**, if a child's value $\geq \beta$, MIN would never let the game reach this node, so further exploration is pointless — prune.
- **Effect:** with good move ordering, effective branching factor drops from b to roughly $\sqrt{b}$, meaning **the same time budget allows searching twice as deep** — a substantial strength increase without extra hardware cost.
- **Move ordering matters most:** trying likely-best moves first (e.g., captures, threats) maximizes early cutoffs, so techniques like **killer-move heuristics** or **transposition tables** (caching previously seen positions) further boost pruning efficiency — especially valuable on constrained hardware.

iv. Evaluation Function Design

Since full-depth search to terminal states is infeasible, a heuristic evaluation function scores non-terminal board states at the depth cutoff. Suggested factors for a checkers-like game:

Factor	Rationale	Relative Weight
Material count (pieces/kings)	Core measure of positional strength	High
King status (promoted pieces)	Kings are more powerful/mobile	High
Piece mobility (legal move count)	More options = more flexibility/attack potential	Medium
Board control / central positioning	Central squares offer more strategic options	Medium
Defensive structure (back-row control)	Prevents opponent promotions	Medium
Threat/capture potential	Immediate tactical opportunities	Medium-high
Piece safety (vulnerability to capture)	Avoid unnecessary losses	Medium

Score = $\sum(w_i \times \text{factor}_i)$, with weights tuned via testing/self-play so the function reasonably approximates true minimax value without deep search.

v. Strategy for Balancing Optimality and Real-Time Performance

1. **Depth-limited Minimax + Alpha-Beta pruning** as the baseline, since full-tree search is infeasible on tablets.
2. **Iterative deepening:** search depth 1, then 2, then 3... within the time budget, always keeping the best move found so far ready — ensures a legal move is available even if time runs out mid-search.
3. **Move ordering heuristics** (captures first, transposition table lookups, killer moves) to maximize Alpha-Beta pruning effectiveness.
4. **Transposition tables:** cache previously evaluated board states to avoid recomputation, valuable given limited CPU.
5. **Quiescence search:** extend search slightly beyond the cutoff only in "unstable" positions (e.g., mid-capture sequences) to avoid misjudging volatile states, without blowing the time budget generally.
6. **Adaptive depth based on device performance/battery state:** dynamically reduce search depth on lower-power conditions to preserve responsiveness.
7. **Accept satisficing over perfect play:** a fast, good move within the time limit is preferable to a theoretically perfect one that misses the turn deadline — this is the

central justification for combining bounded-depth Minimax with Alpha-Beta pruning rather than attempting exhaustive search.